Machine Learning Assignment 2

DRAFT — NOT READY FOR SUBMISSION. Replace placeholders and add your genuine BITS Virtual Lab screenshot. This reference is for learning only.

Student name: [INSERT YOUR NAME] | BITS ID: [INSERT YOUR BITS ID]

1. GitHub Repository Link (REQUIRED FIRST)

https://github.com/YOUR-ACCOUNT/YOUR-REPOSITORY

Confirm the repository contains source code, requirements.txt, README.md, test_data.csv, model/ saved files, and preprocessing metadata.

2. Live Streamlit App Link (REQUIRED SECOND)

https://YOUR-APP.streamlit.app

The URL must open an interactive app that evaluates a CSV without retraining models.

3. BITS Virtual Lab Execution Screenshot (REQUIRED THIRD)

PLACE YOUR AUTHENTIC BITS VIRTUAL LAB SCREENSHOT HERE

Screenshot should show training / assignment execution in BITS Lab

4. README Content (REQUIRED FOURTH)

Classification Comparison Lab — Learning Reference

> **Academic integrity:** This is an educational reference project, not a finished student submission. Run it in the BITS Virtual Lab, review the implementation, document and commit your own experiments, personalize the interface and observations, and use only verified links/screenshots in your PDF.

a. Problem statement

Compare five supervised classification algorithms on one publicly available binary dataset using a consistent hold-out evaluation procedure, then provide an interactive application that evaluates uploaded labeled test data with already-trained models.

b. Dataset description

- Dataset: Wisconsin Diagnostic Breast Cancer (WDBC)

- Original source: UCI Machine Learning Repository — https://archive.ics.uci.edu/dataset/17/breast+cancer+wisconsin+diagnostic

- Access method: sklearn.datasets.load_breast_cancer, which packages the UCI dataset

- Samples: 569; predictor columns: 30; outcome classes: benign and malignant

- Target coding used here: 0 = benign; 1 = malignant

- Split: stratified 80% training / 20% test, random_state=42

- Preprocessing: standardization within Logistic Regression, KNN and SVM pipelines; tree and Gaussian NB use unscaled inputs. Pipelines are fitted only on the training split.

- **Your action:** Explain why you chose this dataset and examine the real class distribution in your BITS Lab run.

c. GitHub repository link

REPLACE AFTER YOU PUBLISH: https://github.com/YOUR-ACCOUNT/YOUR-REPOSITORY

d. Models used and comparison table

1. Logistic Regression — standardized linear classifier.

2. Decision Tree — depth-constrained tree classifier.

3. K-Nearest Neighbors — distance-based classification on standardized features.

4. Gaussian Naive Bayes — conditional Gaussian feature likelihoods.

5. Support Vector Machine — RBF SVC with probability estimates.

Run python train_models.py in BITS Virtual Lab. The actual numerical results are in comparison_metrics.csv. Copy the verified table into this section and into your final PDF; do not fabricate metrics.

| ML Model | Accuracy | AUC | Precision | Recall | F1 | MCC |

|---|---:|---:|---:|---:|---:|---:|

| Logistic Regression | fill from run | | | | | |

| Decision Tree | fill from run | | | | | |

| KNN | fill from run | | | | | |

| Gaussian Naive Bayes | fill from run | | | | | |

| Support Vector Machine | fill from run | | | | | |

Observations (complete from verified results)

| Model | Your observation |

|---|---|

| Logistic Regression | Compare recall and precision; describe possible linear separability. |

| Decision Tree | Compare train/test behavior and influence of max_depth=5. |

| KNN | Discuss scaling, neighborhood size, and test performance. |

| Gaussian Naive Bayes | Discuss conditional independence approximation and observed performance. |

| Support Vector Machine | Discuss its RBF boundary and trade-off among recall, F1 and MCC. |

| Overall winner | State the criterion you choose (e.g., malignant-class recall or F1), show numerical evidence, and justify it. |

Evaluation definitions

Accuracy = correct predictions / total. AUC = ROC area using probability scores. Precision, recall and F1 are computed for the **malignant (positive)** class. MCC measures overall prediction-label correlation and accounts for all confusion-matrix cells.

Project structure

- train_models.py — UCI data loading, stratified split, model training, six-metric evaluation and artifact export
- app.py — test CSV upload, model picker, evaluation, confusion matrix and cross-model comparison
- requirements.txt — required packages
- test_data.csv — held-out labeled test rows generated by the training script
- comparison_metrics.csv — metrics table generated by the training script
- model/*.joblib — saved trained pipelines
- model/metadata.json — feature order and target mapping

Running and deploying

bash

pip install -r requirements.txt

python train_models.py

streamlit run app.py

After verifying everything in the BITS Virtual Lab, create GitHub commits recording meaningful changes, upload the source/model/CSV files, and deploy from https://share.streamlit.io/ using repository main and entrypoint app.py.

REPLACE AFTER DEPLOYMENT: https://YOUR-APP.streamlit.app

What to personalize and verify

- Explain the dataset and class distribution in your own words.
- Try an alternative hyperparameter for at least one model and record the difference.
- Customize page title, section descriptions and chart or table presentation to make your UI your own.
- Interpret model outcomes using **your actual** six-metric comparison.
- Capture one genuine screenshot of assignment execution on BITS Virtual Lab.
- Confirm the public app successfully accepts test_data.csv and returns metrics without retraining.